STAYEASE

Software Requirements Specification

Hotel Room Reservation System

Version	1.1 — consolidated planning decisions
Status	Baseline for review before implementation
Delivery approach	Reference application → verification → trainee starter code
Scope target	Training-focused; approximately 5–8 days

Contents

1. Document Control
2. Purpose and Scope
3. Technology Baseline
4. System Architecture
5. Actors and Authorization
6. Authentication and Session Requirements
7. Hotel Search and Availability
8. Pricing and Tax
9. Reservation and Payment Workflows
10. Cancellation, Refund, Amendments, and No-show
11. Check-in, Check-out, and Physical Rooms
12. Reviews
13. Invoice
14. Admin Operational Dashboard
15. Detailed Audit Trail
16. Inter-service Resilience and Consistency
17. API and Data Contract Requirements
18. Non-functional Requirements
19. Core Acceptance Scenarios
20. Delivery and Verification Phases
21. Open Implementation Details to Freeze Before Coding

1. Document Control

Project: StayEase — Hotel Room Reservation System

Document: Software Requirements Specification (SRS)

Version: 1.1 — Consolidated decisions for review

Status: Baseline for reference implementation; not a claim of completed implementation

Audience: Educator, trainees, reviewers, and implementation agent

Target delivery: Training-focused reference application intended to be implementable in approximately 5–8 days, followed by verification and trainee starter-code preparation

2. Purpose and Scope

- StayEase is a multi-hotel reservation application. Guests can register, search hotels, view room-type availability, reserve rooms, simulate payments, manage eligible reservations, self-check-in/check-out, view printable invoices, and submit verified reviews. Administrators manage hotel operations, view the operational dashboard, manage guest-account support, mark no-shows, and inspect detailed audit records.
- The project must first be implemented as a complete reference application. It must then be verified and defects fixed before the trainee starter-code phase begins. The starter-code phase must not be treated as the first implementation.
- Out of scope: real payment gateway, email delivery, email-based password reset, refresh-token rotation, a fifth Audit/Payment service, dedicated search engine, downloadable PDF generation in the backend, CSV report export, full accounting/folio subsystem, hotel-specific administrator roles, account suspension, login lockout/rate limiting, occupancy supplements, long-stay discounts, and guest approval workflow for reservation amendments.

3. Technology Baseline

- Backend: Java 21; Spring Boot 3.x; Spring Web/REST; Spring Data JPA/Hibernate; Bean Validation; Lombok; Spring Security; JWT access tokens; Maven.
- Microservices: Spring Cloud Gateway on port 8080; Consul service discovery on port 8500; four business services on ports 8081–8084; WebClient with load-balanced discovery; Resilience4j circuit breakers/timeouts where configured.
- Frontend: Angular 16.2.x baseline, TypeScript, HTML5, CSS, Bootstrap 5, Angular Router, reactive forms, HTTP interceptors, and route guards.
- Persistence: MySQL 8.x, one database per business microservice. Suggested database names: stayease_user_db, stayease_property_db, stayease_reservation_db, stayease_review_db.
- Testing and quality: JUnit 5, Mockito, Spring Boot test support, Postman API collection, SonarQube analysis.
- Ports: Gateway 8080; Consul 8500; User Service 8081; Property Service 8082; Reservation Service 8083; Review Service 8084; Angular development server 4200.

4. System Architecture

- User Service (8081): registration, login, user profile, email change, password reset support, role/token-version data, and user-owned audit events.
- Property Service (8082): hotel/property details, room types, physical rooms, amenities, base rates, weekend/seasonal rate overrides, sellable inventory, housekeeping/maintenance status, search filtering, and property-owned audit events.
- Reservation Service (8083): reservation lifecycle, room-type capacity reservation, simulated payments, payment deadlines, cancellation/refund calculation, amendments, check-in/check-out, physical-room assignment references, no-show processing, pricing snapshots, invoice API, operational metrics, and reservation-owned audit events.
- Review Service (8084): verified review creation, published ratings, administrative review removal, and review-owned audit events.
- API Gateway routes client traffic to services and participates in request authentication/routing. Consul provides discovery. Inter-service requests must use service discovery rather than hardcoded hostnames.
- Each service owns its schema and audit records. No service may directly query another service's database. No fifth microservice will be introduced.
- Critical operations fail closed when required dependencies are unavailable. Dashboard and audit aggregation may show partial results only with a visible warning identifying unavailable sources.

5. Actors and Authorization

- Guest (USER): public registration; sign in; maintain permitted profile fields; search hotels; reserve room types; simulate payment; view own reservations and invoices; cancel/amend eligible reservations; self-check-in/check-out; submit one verified review per

completed reservation within 30 days.

- Administrator (ADMIN): shared administration across all hotels; search/view guest accounts; initiate password resets; manage hotel/room/rate/housekeeping data within implemented admin workflows; mark eligible reservations no-show; view operational dashboard; remove inappropriate reviews; view filtered audit history.
- Public registration must always assign USER on the backend. Clients must not choose roles. Initial ADMIN is created through controlled SQL seed/setup.
- Authorization must be enforced in backend endpoints. Angular route guards and hidden buttons are usability controls, not security boundaries. Guests may only access their own profiles, reservations, and invoices.
- Password hashing must use a secure one-way encoder. Never return password hashes or store passwords, JWTs, or reset credentials in logs/audit records.

6. Authentication and Session Requirements

- Login identifier is email. Registration requires full name, unique email, phone number, and password; address is optional.
- Use Spring Security and signed JWT access tokens. Include subject/user identifier, role, token version, issue time, and expiry as appropriate. Validate signature, expiry, and token version for protected requests.
- Each user has a tokenVersion. Increment it after password reset and successful email change. Protected requests compare the token's version with the current version. Logout clears the current browser token; server-side token-version invalidation handles security-sensitive changes.
- Email changes require an authenticated guest's current password and a unique valid new email. On success, increment tokenVersion and require a fresh login. No email verification service is included.
- Password recovery is administrator-assisted. Administrators can search/view guest accounts and initiate a reset only; they cannot edit guest profiles or activate/deactivate/suspend accounts.
- Basic login validation only: generic credential failure message; no persistent failed-attempt counter, account lockout, or login rate limiting in this scope. This limitation must be documented.

7. Hotel Search and Availability

- Search supports hotel name, destination, check-in/check-out dates, guest count, price range, room type, minimum rating, and amenities filters.
- Use standard page-number pagination with page, size, total elements, and total pages. Filters must persist while navigating pages.
- Availability is calculated by room type for every night of the requested stay. Search and booking must not rely on a check-in-date-only check.
- Hybrid inventory: room-type inventory governs search and reservation capacity; individual physical rooms are used for housekeeping and assigned at check-in.
- Bookings cannot be confirmed unless inventory for the full stay is secured. A physical room being dirty affects immediate check-in readiness, but does not by itself imply the hotel has no sellable room-type capacity for future dates. Out-of-service rooms must not count as sellable capacity.
- Search/booking must fail clearly if authoritative availability cannot be obtained; no stale cached inventory may be used to confirm a reservation.

8. Pricing and Tax

- Each room type has a base nightly rate. Weekend and seasonal overrides may apply per night.
- Rate precedence for a date: seasonal override first; otherwise weekend override; otherwise base rate. This rule resolves overlap deterministically.
- Apply one fixed configurable tax percentage to the eligible subtotal. No occupancy supplements or long-stay discounts.
- At booking, persist a price snapshot: nightly rates by date, number of nights, subtotal, tax percentage, tax amount, and total. Invoice and cancellation calculations use this snapshot rather than current room rates.
- Currency is INR as per the source project baseline. Monetary values must use decimal-safe types (e.g., BigDecimal) and explicit rounding rules.

9. Reservation and Payment Workflows

- Reservation status and payment status are separate concepts. Suggested reservation states: PENDING_PAYMENT (for prepaid attempt/temporary hold), CONFIRMED, PAYMENT_FAILED, CANCELLED, CHECKED_IN, CHECKED_OUT, NO_SHOW. Payment states: PENDING, PAID, FAILED, REFUNDED. Final enum design must avoid contradictory states and be documented in API contracts.

- Prepaid: validate availability and create a bounded temporary inventory hold; attempt simulated payment; confirm only after payment succeeds and inventory reservation remains secured. If payment fails, leave unconfirmed/failed and release the hold. If the result is uncertain due to timeout, reconcile idempotently before releasing inventory or retrying.
- Pay-later: validate availability, secure room-type capacity, and confirm with payment PENDING. Payment must occur before the earlier of booking time + 24 hours or scheduled check-in time. If unpaid at deadline, cancel with reason PAYMENT_TIMEOUT and release inventory.
- No available inventory means no confirmed reservation. Inventory reservation/release must be idempotent and must not double-release or double-reserve capacity.
- Simulated payment methods may include CARD, UPI, and NETBANKING. No external provider or real money movement is involved. Duplicate payment requests must be safely handled with idempotency keys or equivalent persisted request identity.
- Service failure: payment must not be reported as successful unless its result is persisted. A caller timeout is an uncertain outcome, not proof of failure.

10. Cancellation, Refund, Amendments, and No-show

- Fixed cancellation policy: cancellation at least 24 hours before scheduled check-in is free; cancellation less than 24 hours before check-in retains one night's charge. Apply only to eligible guest-initiated cancellations and paid amounts; simulate any eligible refund.
- Payment-timeout cancellation is a separate system cancellation reason and does not incur the guest-initiated late-cancellation penalty.
- Cancellation is idempotent; completed stays cannot use ordinary cancellation. Record reason, penalty, refund amount, timestamps, and audit event.
- Guests can amend their own eligible reservations; administrators can amend eligible reservations across the system. Standard amendments are allowed before check-in only; cancelled, no-show, checked-in, and checked-out reservations cannot be amended through the standard workflow.
- Amendments support dates and room type, validate inventory, reprice using current applicable rates, and store a revised price snapshot. Reconcile the new total against payment already collected by simulating additional payment or eligible refund. Record before/after values in audit.
- Admin manually marks eligible reservations NO_SHOW. Reject if already checked in, checked out, cancelled, or already no-show. Do not invent a separate no-show penalty; apply only the agreed policy where applicable. Release unused inventory for applicable dates.

11. Check-in, Check-out, and Physical Rooms

- Physical rooms are assigned at check-in, not at booking confirmation.
- Guest self-service initiates check-in. Backend verifies ownership, confirmed status, scheduled date eligibility, payment PAID, and an eligible physical room of the booked room type. If no eligible room is ready, reject check-in and leave the reservation CONFIRMED.
- Physical room must not be DIRTY, INSPECTING, or OUT_OF_SERVICE and must not overlap another active physical assignment. Eligible room state must be validated against Property Service.
- Successful check-in records room assignment, changes reservation to CHECKED_IN, and creates an audit event.
- Guest self-service check-out is allowed only for the guest's own CHECKED_IN reservation. Successful checkout changes status to CHECKED_OUT, records checkout time, and changes the assigned physical room to DIRTY for housekeeping.
- Housekeeping statuses: CLEAN, DIRTY, INSPECTING, OUT_OF_SERVICE. A room can become assignable only after the allowed housekeeping transitions clear it.

12. Reviews

- Only authenticated guests may review their own reservation after CHECKED_OUT, within 30 days of checkout. One review per reservation.
- Valid reviews publish immediately. Administrators can remove inappropriate reviews. No pre-publication moderation queue.
- Review Service must verify reservation ownership and checkout eligibility through Reservation Service. If that service is unavailable, reject review submission rather than trusting client-provided IDs.
- Review creation and administrative removal must be audited. Rating range and text limits must be validated and documented.

13. Invoice

- Basic invoice rendered on demand as printable HTML in Angular; browser print/save-to-PDF is used, with no backend PDF-generation dependency.

- Invoice contains hotel and guest details, reservation ID, stay dates, nightly charges, subtotal, tax percentage/amount, total, and persisted payment status.
- Guests can view invoices only for their own reservations. Administrators may view authorized reservations across the system.
- Invoice rendering does not change reservation/payment state. Use stored price snapshot and persisted payment data. No separate invoice-finalization lifecycle, folio, credit note, or void/reissue workflow.

14. Admin Operational Dashboard

- Show reservation counts by status; today's check-ins/check-outs; occupancy percentage; room availability; pending payments; reservation revenue; cancellations; no-shows; and housekeeping/maintenance status.
- Metrics must be based on persisted data and clearly defined date ranges. Revenue must distinguish successful payments from pending/failed payments.
- Occupancy must be calculated from relevant sellable inventory and stay dates, not simply reservation count.
- No CSV export in the current scope.
- If one or more non-critical sources are unavailable, display available metrics with a visible partial-data warning; do not present partial results as complete.

15. Detailed Audit Trail

- Each service stores its own audit records in its own database; no central audit database or fifth service.
- Record actor, action/event category, entity ID, timestamp, concise description, and before/after values for amendments, payment changes, room status changes, and moderation actions where applicable.
- Audit events cover user/account security events, reservation lifecycle, payments/refunds, pricing snapshots/invoice-related events, room operations, reviews, and administrative actions.
- Audit history is administrator-only and supports filters for date range, event category, actor, and entity ID. Retrieve through service APIs; never query service databases directly.
- If a source service is unavailable, show available audit records with a prominent warning that the history is incomplete.
- Never store passwords, JWTs, or reset credentials in audit records. Business audit records should reflect successful state changes; failed attempts may be logged separately as security telemetry if explicitly designed.

16. Inter-service Resilience and Consistency

- Fail closed for booking, payment, check-in/check-out, amendments, and review eligibility checks when required dependencies are unavailable.
- Non-critical dashboard/audit screens may show partial results with explicit warnings.
- Use service discovery, bounded connection/read timeouts, consistent error response format, and Resilience4j circuit breakers where configured.
- Do not use stale cached inventory or pricing to confirm bookings. A timeout is not necessarily a failed operation; use idempotency and reconciliation for payment/booking operations.
- Each service owns its database. Cross-service operations cannot rely on a single ACID transaction across multiple databases; define explicit reservation/hold states, idempotent compensation, and clear failure recovery.
- Reservation capacity is authoritative in Reservation Service reservation records or a well-defined inventory reservation ledger; Property Service provides sellable inventory and room eligibility data. The final implementation blueprint must define the API ownership to avoid circular service calls.

17. API and Data Contract Requirements

- REST APIs must use DTOs rather than exposing JPA entities. Validate request DTOs with Bean Validation. Use consistent success/error response shapes and meaningful HTTP status codes.
- Protected endpoints must authenticate JWT and enforce role and ownership rules. Service-to-service calls must be authenticated/trusted according to the implementation's chosen configuration; do not trust client-supplied actor identity.
- Provide OpenAPI/Swagger documentation for service endpoints and a Postman collection for key workflows.
- Use decimal-safe monetary fields, ISO date/date-time formats, explicit timezone assumptions, pagination metadata, and documented enum values.
- API contracts must specify idempotency behavior for payment, cancellation, timeout processing, and inventory release.

18. Non-functional Requirements

- Security: backend JWT validation, role/ownership checks, secure password hashing, input validation, no secret leakage, token version invalidation.
- Reliability: fail-closed critical operations, idempotent retries, explicit partial-data indicators, bounded service calls.
- Maintainability: service ownership boundaries, DTOs, layered architecture, centralized exception handling per service, consistent naming and configuration.
- Usability: responsive Angular UI, accessible labels, actionable validation/error messages, visible payment and reservation statuses, printable invoice.
- Performance: pagination for list endpoints; avoid unbounded data retrieval; define test datasets for search and dashboard aggregation.
- Compatibility: supported modern desktop browsers; Angular frontend on port 4200 in development.
- Quality: automated unit/integration tests, Postman smoke tests, SonarQube analysis, and a documented known-limitations section.

19. Core Acceptance Scenarios

- Registration cannot create ADMIN; duplicate email is rejected; passwords are hashed; login errors are generic.
- JWT signature/expiry/tokenVersion are enforced; password reset/email change invalidates old tokens; guest cannot access another guest's reservation or invoice.
- Search filters and pagination work together; availability covers all nights; no inventory means no confirmed booking.
- Prepaid payment success confirms; failure releases hold; uncertain result is reconciled safely; pay-later deadline is the earlier of 24 hours after booking or check-in.
- Payment-timeout cancellation releases capacity once; cancellation penalty/refund matches fixed policy and booking price snapshot.
- Amendments validate availability, reprice, reconcile payment, and capture before/after audit data.
- Self-check-in requires payment and an eligible room; no ready room rejects check-in without changing confirmed status; checkout marks room DIRTY.
- No-show is manually admin-triggered with status validation and inventory release.
- Review requires own checked-out reservation within 30 days and is limited to one per reservation; admin removal is authorized and audited.
- Invoice is on-demand printable HTML and uses persisted snapshot/payment status.
- Dashboard and audit aggregation indicate partial data if a service is unavailable.
- Service-owned audit events are persisted and admin audit filters return correct results.

20. Delivery and Verification Phases

- Phase 1 — Baseline: approve this SRS/blueprint; freeze API/data contracts; define service boundaries and workflow states.
- Phase 2 — Complete reference implementation: build four backend services, gateway, Angular UI, schemas/seed scripts, and test artifacts. Do not claim completion before tests run.
- Phase 3 — Verification: run unit/integration tests, API workflow tests, database validation, frontend smoke tests, security/ownership checks, and SonarQube; fix defects and record remaining limitations.
- Phase 4 — Trainee starter code: only after reference implementation is verified, derive starter code, trainee tasks, acceptance criteria, and educator notes.
- Manual infrastructure constraint: user manually initializes MySQL schemas/data and starts MySQL/Consul. Agent must not start/stop MySQL or Consul. After SQL changes, notify user to initialize scripts manually before dependent integration work continues.

21. Open Implementation Details to Freeze Before Coding

- Exact reservation status enum and prepaid temporary-hold expiry duration.
- Exact API contract for room-type capacity reservation/release and Property/Reservation service call direction; avoid circular nested calls.
- How physical room counts contribute to sellable room-type inventory when out-of-service. Use a clearly defined sellable-capacity field or derived rule, documented and tested.
- Exact fixed tax percentage default and monetary rounding mode. Keep percentage configurable.
- JWT signing configuration, expiry duration, secret management for local development, and trusted service-to-service authentication approach.

- Admin bootstrap mechanism and safe local-development credentials handling.
- Audit aggregation strategy (Angular requests each service directly through Gateway or a backend aggregation endpoint); choose one and implement partial-failure warnings.
- These are implementation-level details; they must be documented explicitly and must not silently override the 35+ business decisions recorded during planning.